Is Folding Worth the Storage?
Curved and Shrinking Space-Time Gaussians for Video

Name: Chen Hongshan
Student ID: A0311136W

1. Summary

This project tests whether the two "folding" extensions of **space-time Gaussians** proposed by *VeGaS*, a curved (quadratic) trajectory and a shrinking of each slice as it fades, still improve video fitting once storage is held equal. I plan to extend the space-time Gaussian fit of Unit 16 on the UVG Jockey clip, compare four models in a 2×2 ablation at four storage budgets, and measure where in the frame any gain comes from.

2. Research question

At an equal number of stored parameters, does adding a quadratic acceleration term, a fade-dependent shrinking of the slice covariance, or both, to the space-time Gaussians of Unit 16 raise the mean PSNR of the fitted UVG Jockey clip (10 and 30 frames at 480×270)? If either extension helps, is the gain concentrated in the region of non-uniform motion (the horse) rather than in the panning background?

3. Motivation and relation to the course

Unit 16 represents a video by Gaussians in space and time. Slicing one at time t gives a 2D Gaussian whose centre moves at a constant velocity, whose shape is constant, and whose weight fades away from its peak time. Each Gaussian costs 12 stored numbers however many frames it spans, but a patch that swings or changes size, such as the leg of a galloping horse, which swings forward and back and changes its apparent length within a few frames, needs several short-lived Gaussians.

VeGaS (Smolak-Dyżewska et al., 2024) generalises this with Folded-Gaussians. The slice centre follows a polynomial in time (degree 7 by default), and the slice covariance is scaled by the time likelihood, so a Gaussian also shrinks as it fades. The paper reports higher reconstruction PSNR on DAVIS than NeRV-family models and VGR, and its main goal is realistic video editing.

The paper, however, does not show that folding is worth its storage. Its ablation (Table 3 of the paper) uses *Big Buck Bunny*, an open-source animated short film of which the paper uses 132 frames at 720×1280, and varies the polynomial degree from 1 to 9: at batch size 3, degree 1 reaches 38.15 dB with 0.57M Gaussians and degree 7 reaches 38.53 dB with 0.62M. A higher degree adds coefficients to every Gaussian and the final counts differ, so the gain cannot be attributed to the curved trajectory itself. Shrinking is never ablated separately, no model size is reported, and all runs use hundreds of thousands of Gaussians.

The answer is also not obvious from the course. Units 14 and 15 judge a representation by its

quality at a given size. The acceleration term costs 2 extra numbers for every Gaussian, including the background Gaussians, which drift with the near-uniform camera pan described in Unit 16 and do not need to curve. Under a fixed budget, fewer Gaussians that can curve therefore compete with more Gaussians that move only in straight lines, and either could win.

The question builds on five units of the course: Unit 1 (conditioning of a Gaussian, Cholesky factors), Unit 5 (gradients of an image loss), Unit 6 (PSNR, SSIM and the evaluation protocol), Unit 14 (equal-budget comparisons and rate–distortion) and Unit 16 (the representation being extended).

4. Problem formulation

I plan to compare four models that differ only in how a space-time Gaussian is sliced at time t. All four render a frame as the accumulated sum of Unit 16, clamped to [0, 1]:

$$C_t(p) = \Sigma_i\, w_i(t) \cdot c_i \cdot \exp\!\left(-\tfrac{1}{2}\,(p-\mu_i(t))^T\, \Sigma_i(t)^{-1}\,(p-\mu_i(t))\right), \quad w_i(t) = \exp\!\left(-(t-t_{0,i})^2 / (2\,\sigma_{t,i}^2)\right)$$

Each Gaussian stores its centre μ (2 numbers), peak time t_0 (1), duration σ_t (1), velocity v (2), the Cholesky factor L of its slice covariance (3) and colour c (3), as in the Unit 16 example. Writing $\tau = t - t_0$:

Model	Slice centre μ(t)	Slice covariance Σ(t)	Numbers per Gaussian
M0 baseline (Unit 16)	$\mu + v\tau$	$L\,L^T$	12
M1 acceleration	$\mu + v\tau + a\tau^2$	$L\,L^T$	14
M2 shrinking	$\mu + v\tau$	$\max(w(t), s_{min}) \cdot L\,L^T$	12
M3 both	$\mu + v\tau + a\tau^2$	$\max(w(t), s_{min}) \cdot L\,L^T$	14

The acceleration *a* adds 2 numbers per Gaussian. Shrinking adds none; the floor s_{min} keeps the covariance invertible as the weight approaches zero.

Equal storage.
For a budget of B stored numbers, M0 and M2 use $\lfloor B/12 \rfloor$ Gaussians and M1 and M3 use $\lfloor B/14 \rfloor$. All parameters are float32 and none are quantised, so every number takes 4 bytes and an equal count of stored numbers is also an equal file size. I report sizes in both numbers and megabytes. For the 10-frame clip, $B \in \{3{,}000; 6{,}000; 12{,}000; 24{,}000\}$, which is 250, 500, 1,000 and 2,000 Gaussians of M0, the counts used in the Unit 16 example. For the 30-frame clip the budgets are tripled, so each frame receives the same budget.

Held fixed.
The renderer, the loss (mean squared error over all frames, fitted jointly), the Adam optimiser and learning rates of shared parameters, the iteration count, the 480×270 resolution, and the random seed and initialisation of shared parameters. The acceleration starts at a = 0, so M1 is identical to M0 at initialisation. There is no densification or pruning, so every model keeps exactly its budgeted number of Gaussians.

Swept hyperparameters,
chosen on a validation clip (see Evaluation plan): the shrink floor $s_{min} \in \{0.05, 0.1, 0.2\}$ for M2 and M3, and the learning rate of a $\in \{0.3, 1, 3\}$ × the learning rate of v for M1 and M3. The budget B is swept as an experimental variable, and every configuration is run with 3 random seeds.

5. Planned implementation

The two extensions change only the slicing step; the renderer is the same image renderer for all four models. I therefore write the slicing step myself and reuse the course's 2D renderer and its hand-written gradients.

What exists already

- The Unit 16 formulation of space-time Gaussians and the course program gaussian_image_video.py. It provides a Warp renderer that sums 2D Gaussians, a hand-written backward pass for each Gaussian's centre, Cholesky factor and colour, an Adam fitting loop, and a finite-difference gradient check. It fits images and predicts video frame by frame, but has no space-time Gaussians.
- The GaussianImage paper and code (Zhang et al., ECCV 2024), the source of the accumulated-sum rendering; its CUDA rasteriser is a fallback only.
- The UVG Jockey sequence (Mercat et al., 2020).

What I will write

- A slicing step for M0-M3 that maps each Gaussian and a time t to its 2D centre, Cholesky factor and time-weighted colour, which are passed to the course's render and gradient kernels.
- The joint fitting loop over all frames of a clip, with logging of loss, PSNR and time.
- The chain rule from the per-frame 2D gradients that the course kernels return to the space-time parameters (μ, v, a, t_0, σ_t, L, c), summed over frames, including the derivatives of the acceleration and shrinking terms. I will check it against finite differences by extending the course's gradient check.
- Evaluation scripts for per-frame, per-region and per-seed metrics, plots and crops, and hand-drawn horse masks for every frame.
- Speed fallback: if the Warp renderer is too slow for the 96 fits, I will switch to the GaussianImage CUDA rasteriser, which computes the same accumulated sum.

Data.
Jockey reduced to 480×270, as in Unit 16. The course program loads Lego views, so I will write a loader for the Jockey frames.

- Test clips: frames 0–9 (comparable with the notes) and frames 0-29.
- Validation clip for choosing hyperparameters: a 10-frame segment of Jockey that I will select after inspecting the sequence. It must not overlap frames 0-29, and it must show content similar to the test clips (the galloping horse with the camera panning), so that settings chosen on it transfer to the test clips.

Compute.
One GPU of the SoC Compute Cluster (an NVIDIA A100), with runs submitted as a Slurm job array. The test runs total 4 models × 4 budgets × 2 clips × 3 seeds = 96 fits, plus the validation sweeps.

6. Evaluation plan

Comparison protocol. The primary comparison is at equal stored parameters and equal

iterations. Following Unit 14, the swept hyperparameters are chosen by PSNR on the validation clip, and each chosen configuration is then reported once on the two test clips. Each result is the mean over 3 seeds with its min-max range.

Metrics

Metric	What it tells me
Mean PSNR vs stored numbers (one curve per model)	The main answer: which model gives the best quality for a given storage
SSIM	Whether a PSNR gain reflects real structure or only smoother colour
PSNR inside vs outside the horse mask	Whether a gain comes from non-uniform motion or from the background
Per-frame PSNR and per-frame difference to M0	Whether a mean hides frames where a model fails

Qualitative and diagnostic evidence

- The worst frame of each model, with magnified crops of the horse's legs and of the left edge where new background enters.
- Error maps of the same frames.
- The distribution of the fitted accelerations, including the share of Gaussians whose acceleration moves the centre by less than half a pixel over its lifetime: a direct measure of storage spent on Gaussians that do not use it.
- For M2 and M3, crops that check whether shrinking leaves visible gaps.

Costs reported alongside quality.
Stored numbers and bytes per model, training time per iteration and in total, rendering time per frame, and peak GPU memory.

What would show the idea did not work.
I will conclude that the acceleration is not worth its storage if, at equal storage, M1 is not above M0 by more than 0.3 dB and by more than the seed-to-seed range, at three or more of the four budgets, on both test clips. The same criterion, applied to M2 against M0, decides for shrinking. If a gain appears only inside the horse mask while whole-frame PSNR does not rise, I will report the benefit as local and not paying for its storage.

7. Expected analysis

The final report should answer:

- Does acceleration, shrinking, or both raise PSNR at equal storage, and by how much at each budget?
- Does the answer change with the budget? I expect acceleration to matter most at small budgets, where each Gaussian must cover more motion alone, and possibly to lose at large budgets, where short-lived linear Gaussians can relay.
- Is any gain inside the horse mask, outside it, or both?
- What share of the fitted accelerations is close to zero, that is, how much of the extra storage goes unused?
- Does shrinking leave visible gaps, and how sensitive is it to the floor s_{min}?
- Does the 30-frame clip change the conclusion drawn from the 10-frame clip?
- What do the extensions cost in time and memory?

8. Extension (if time permits)

The slicing module will support a polynomial trajectory of any degree d, at a cost of $12 + 2(d - 1)$ numbers per Gaussian. If the main experiments finish on schedule, I will compare $d \in \{1, 2, 3, 7\}$ at equal storage on the 30-frame clip, which tests whether the higher-degree gain in the VeGaS ablation survives once storage is controlled. I will also render slices at times between frames (for example $t = 4.5$) to check whether high-degree trajectories oscillate between the frames they were fitted to.

9. Risks and limitations

- Shrinking can leave gaps that neighbouring Gaussians must fill, and the covariance can become ill-conditioned; the floor s_{min} and its sweep address this, together with the course renderer's lower bound of 0.3 pixels on the diagonal of the Cholesky factor.
- The acceleration may destabilise early training; it starts at zero and has its own swept learning rate.
- Each iteration renders every frame of the clip, so the Warp renderer may be slow on the 30-frame clip at the largest budget; the GaussianImage rasteriser is the fallback.
- The motion in the 10-frame clip is close to uniform, so acceleration may have little to gain there; the 30-frame clip is included for this reason.
- The conclusions hold only for the Jockey sequence, these budgets and this resolution. I will not claim that folding is better or worse in general.
- The horse masks are hand-drawn and approximate, so region results are indicative rather than exact.

10. Timeline

1. Week 10 (19–25 Oct): submit the proposal; set up the course program on the cluster and write the Jockey loader; implement M0 and reproduce the Unit 16 10-frame fit.
2. Week 11 (26 Oct–1 Nov): implement M1–M3, check gradients, run the validation sweeps.
3. Week 12 (2–8 Nov): run all test configurations, draw the horse masks, run the extension if on schedule.
4. Week 13 (9–13 Nov): analyse the results, give the presentation, submit the report by Friday 13 November.

11. References

- CP4282 course notes, Unit 16: 2D Gaussians for Images and Videos and Unit 14: Compaction of 3DGS.
- W. Smolak-Dyżewska, D. Malarz, K. Howil, J. Kaczmarczyk, M. Mazur, P. Spurek. VeGaS: Video Gaussian Splatting. arXiv:2411.11024, 2024.
- X. Zhang et al. GaussianImage: 1000 FPS Image Representation and Compression by 2D Gaussian Splatting. ECCV 2024. Code.

- A. Mercat, M. Viitanen, J. Vanne. UVG dataset: 50/120fps 4K sequences for video codec analysis and development. ACM MMSys 2020.

12. AI-use note

Claude Opus 5.5 (Anthropic) was used to read the course notes and the VeGaS paper, discuss the experimental design, and draft the wording of this proposal. I made and checked the decisions on the research question, the models compared, the budgets and the failure criteria.